AI Engineering and Claude Architect · Part II, Retrieval

Vector Search, Lesson 4

Choosing a Vector Store

The team spent a month evaluating dedicated vector databases for 3 million chunks. Their data already lived in Postgres, with the access rules, the audit log and the transactions. The most useful question nobody had asked was "do we need a new database at all?"

Module	AI04 of 13
Lesson	4 of 5
Prerequisites	AI04 L1–L3, HLD M21A ("where the vectors live")
Exam relevance	Out of scope
Reading time	~30 minutes

Contents

1. Choosing a Vector Store

Lesson 4 of 5 · Vector Databases and Search

Choosing a Vector Store

Key idea

Choose a vector store by your constraints, not by its feature list: where your data already lives, your scale, your tenancy model and who operates it. Start with the database you already run unless a measured limit says otherwise, because **the hard part is vectors and business data together**, and every new system is one more copy to keep consistent. Decide with a benchmark on your own data, and design so that you can migrate later.

> *Before: HLD M21A argues for starting in Postgres, and gives a RAM-based trigger for moving out. Teaches: the options and what each is good for, a decision method, a pgvector design you can use, and how to benchmark properly. Used by: AI04 L5, AI05 L7, AI08 L5.*

The hook: the wrong first question

A bank's team asks: "Which vector database should we buy?" They compare feature matrices for a month. Meanwhile, their real requirements are these:

Requirement	Detail
Scale	3 million chunks, growing slowly
Data	Documents, permissions and audit already in Postgres
Compliance	Row-level access rules, and a change log
Team	Strong Java and SQL, and no appetite for a new database

> *Key idea*
>
> **At this size, on this data, a new database adds a second copy of the data and a new way to leak it, and solves no problem they have.** The right first question is "what limit are we about to hit, and how would we know?"

At 3 million × 1024 × 4 bytes, the raw vectors are 12.3 GB. That fits on an ordinary database server.

Intuition: a filing cabinet or a warehouse

If you run a small office, a new filing drawer in your existing cabinet beats renting a warehouse. Everything's in one place, and one person manages it. You rent a warehouse when the cabinet is truly full, or when you need forklifts. Renting one early means paying for space, moving things back and forth, and tracking which copy is current.

The landscape

Category	Examples	Natural fit	Main weakness
Relational extension	pgvector	Data already in Postgres, moderate scale, SQL filters	Bound by one Postgres instance's limits, unless partitioned or sharded
Search engine	OpenSearch, Elasticsearch	You already run it, and need hybrid search	Cluster operations, and JVM memory tuning
Dedicated, self-hosted	Qdrant, Milvus, Weaviate	Large scale, advanced filtering, and quantization	A new system to run
Dedicated, managed	Pinecone	No operations, and multi-tenant namespaces	Vendor dependence and usage cost
Library	FAISS, hnswlib	Embedded index, or offline batch work	You build the database around it
Exact search	Plain code	Thousands of vectors	Doesn't scale

I read the pgvector, Qdrant, Pinecone and OpenSearch documentation for this module. Details for Milvus, Weaviate, Elasticsearch, FAISS and hnswlib come from general knowledge and HLD M21A, so check them before deciding.

What each one's documentation emphasizes

Product	Filtering	Multi-tenancy	Hybrid
pgvector	Filters apply after the scan. Iterative scans, partial indexes, partitioning	Your design, with Postgres features	With Postgres full-text search, using RRF or a cross-encoder
Qdrant	Payload filtering	Single collection partitioned by payload, `is_tenant`, tiered shards	Not checked here
Pinecone	Metadata filters, which scan the namespace	One namespace per tenant, with deletion described as nearly instant	Not checked here
OpenSearch	Query DSL	Not checked here	Normalization processor, or RRF score ranker

A decision method

Work through these in order. Stop at the first clear answer.

#	Question	If yes
1	Under a few thousand vectors?	Exact search in code
2	Is your source data in Postgres, and does the index fit in RAM on one reasonable machine?	**pgvector**
3	Do you already run OpenSearch or Elasticsearch for search?	Add vectors there
4	Do you need no operational burden, and accept usage pricing and a vendor?	**Managed**, such as Pinecone
5	Very large scale, complex filters, and a team to run it?	**Dedicated self-hosted**, such as Qdrant or Milvus

Then test the top choice with a benchmark. This order is an engineering recommendation.

When to leave Postgres

HLD M21A gives the trigger: when the index no longer fits comfortably in one machine's RAM, or build and upsert load competes with your transactional workload. Some measurable signals:

Signal	What it means
Buffer cache misses on the index	The index no longer fits in memory
p99 latency rising with index size	Out-of-memory graph walks
Long index builds affecting OLTP	Workloads should separate
Many tenants, with filter recall problems	Partitioning becomes painful

A read replica for vector queries, or a separate Postgres instance, can be a useful step before a new product. It keeps your SQL, your skills and your tools.

A pgvector design for real use

This SQL hasn't been run, since Postgres isn't available in this environment. It follows pgvector's documentation, and you should test it before relying on it.

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE chunks (
    tenant_id     text         NOT NULL,
    id            bigserial,
    document_id   bigint       NOT NULL,
    model_version text         NOT NULL,
    content_hash  text         NOT NULL,
    text          text         NOT NULL,
    source_url    text,
    embedding     vector(1024) NOT NULL,
    tsv           tsvector GENERATED ALWAYS AS
                  (to_tsvector('english', text)) STORED,
    PRIMARY KEY (tenant_id, id)
) PARTITION BY HASH (tenant_id);

CREATE TABLE chunks_p0 PARTITION OF chunks
    FOR VALUES WITH (MODULUS 16, REMAINDER 0);
-- Repeat for remainders 1 to 15.

CREATE INDEX ON chunks USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);
CREATE INDEX ON chunks USING gin (tsv);
```

Design choice	Why
Hash partition by tenant	A search touches one partition, which reduces the correlated filter problem from AI04 L2. Note that a large tenant shares its partition with others, and AI04 L5 refines this
`model_version` on each row	The guard from AI04 L1, and allows `v2` to coexist
`content_hash`	Skip re-embedding unchanged text
`text` and `source_url` stored	Needed for prompts and citations
Generated `tsv` column	BM25-like text search, updated in the same transaction

Enforce tenancy in the database as well as the app:

```
ALTER TABLE chunks ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON chunks
    USING (tenant_id = current_setting('app.tenant_id'));
```

From Java

From `code/ai04-vector/pgvector/.../ChunkRepository.java`, which compiles against `pgvector-java` 0.1.6 and the Postgres driver, but hasn't been run:

```
private static final String SEARCH = """
        SELECT id, text, embedding <=> ? AS distance
        FROM chunks
        WHERE tenant_id = ? AND model_version = ?
        ORDER BY embedding <=> ?
        LIMIT ?""";

public List<Hit> search(String tenantId, String modelVersion, float[] query, int k)
        throws SQLException {
    boolean previous = connection.getAutoCommit();
    connection.setAutoCommit(false);
    try (Statement settings = connection.createStatement()) {
        settings.execute("SET LOCAL hnsw.ef_search = 100");
        settings.execute("SET LOCAL hnsw.iterative_scan = 'relaxed_order'");
        List<Hit> hits = run(tenantId, modelVersion, query, k);
        connection.commit();
        return hits;
    } catch (SQLException e) {
        connection.rollback();
        throw e;
    } finally {
        connection.setAutoCommit(previous);
    }
}
```

Why `SET LOCAL`? It lasts only for the transaction, so a pooled connection doesn't carry your setting to the next request. With a plain `SET`, a pooled connection would leak it, which is a classic Spring Boot and HikariCP surprise. The same concern applies when setting `app.tenant_id` for row-level security: a value left on a pooled connection is a cross-tenant leak.

Gradle dependencies:

```
implementation("com.pgvector:pgvector:0.1.6")
implementation("org.postgresql:postgresql:42.7.14")
```

Benchmark before you commit

Vendor benchmarks show someone else's best case. A useful benchmark reproduces yours.

Do	Why
Use your own vectors	Recall depends on data, as AI04 L2 showed
Use your own filters, including rare and correlated ones	That's where systems differ most
Report latency **at a stated recall**	Latency alone means nothing: anyone can be fast at low recall
Test while writes are happening	Reads alone hide contention
Test cold starts and restarts	Index loading can take a long time
Run p99, not just the mean	Fan-out multiplies tail latency
Estimate total cost, including operations	People cost more than nodes

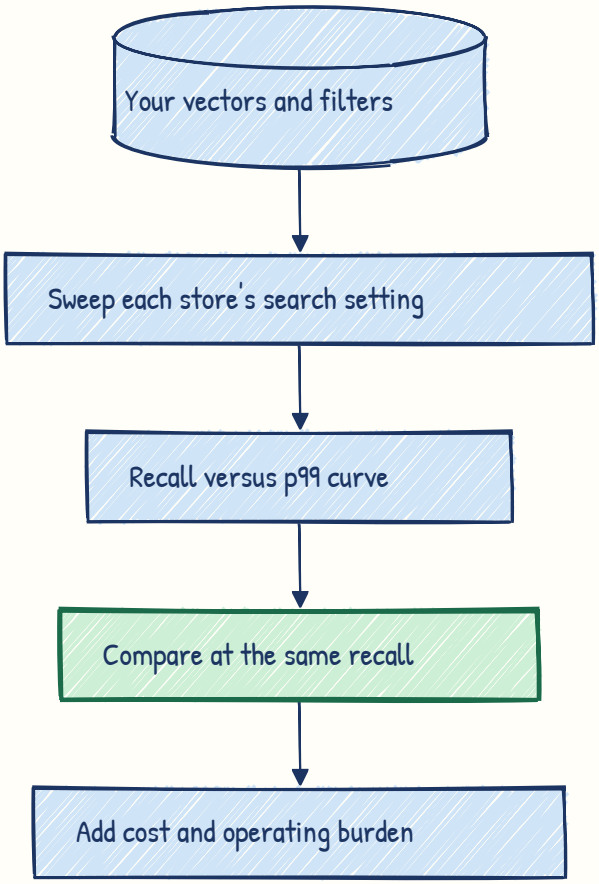


Figure 1. The right chart to produce. Fix a recall target, then compare latency and cost at that target. Every other comparison can mislead.

Stay portable

You may need to migrate one day. A small interface keeps that affordable.

```
public interface ChunkSearch {
    List<Hit> search(String tenantId, String modelVersion, float[] query, int k);
}
```

Also keep raw text and vectors in your own durable store, so the search system is a **rebuildable derived index**. That single decision makes migration a batch job instead of a crisis. **Don't build an elaborate abstraction over every vendor feature**, since LLD's rule applies: only abstract what you actually need to vary.

How the field and Anthropic approach it

Problem	Direction the field took
Vectors separate from business data	Vector support inside existing databases
Running a vector DB is hard	Managed, serverless offerings
Filter recall	Filter-aware graph traversal
Multi-tenancy	Namespaces, and tenant-aware storage

Anthropic doesn't sell a vector store, and it doesn't prescribe one. Its RAG guidance focuses on retrieval quality (AI05 L3). So this decision stays with you.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: a new vector DB on day one. A second copy of your data, with permissions re-implemented, and no way to join.

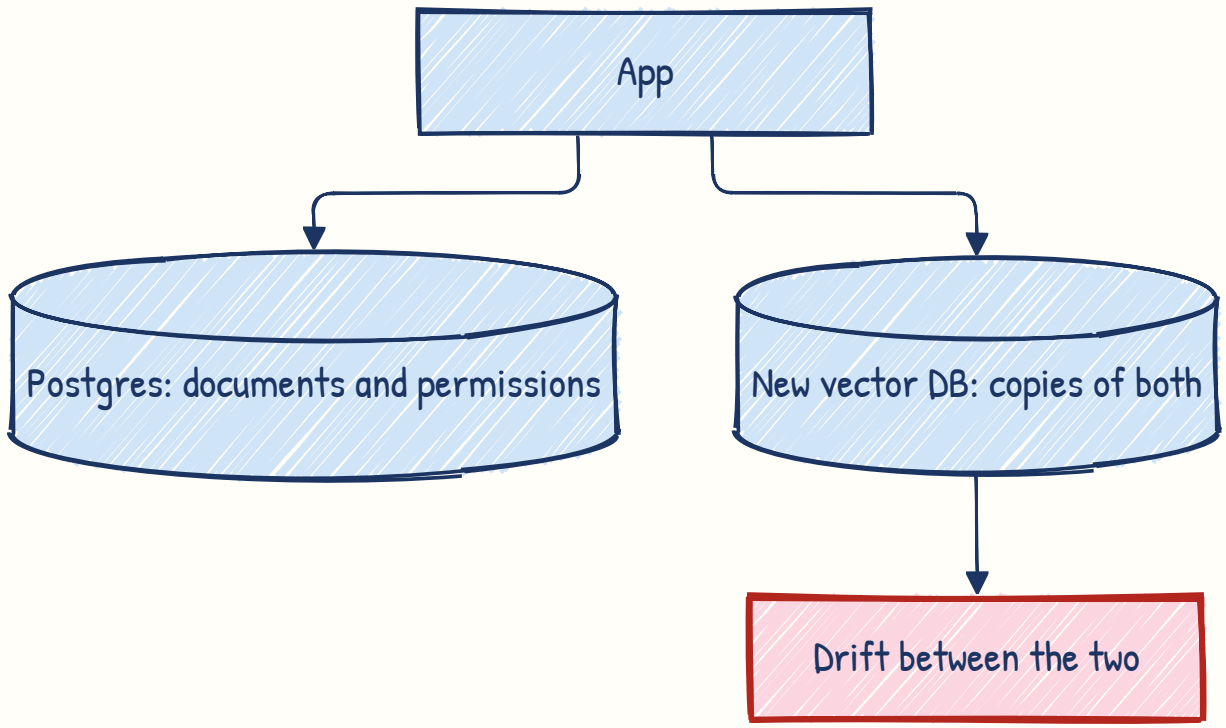


Figure 2. Baseline. The vector store holds a second copy of the data and its permissions, and the two can disagree.

Improved: pgvector in the main database. One copy, with one set of permissions. New problem: **index builds and vector queries now compete with transactions.**

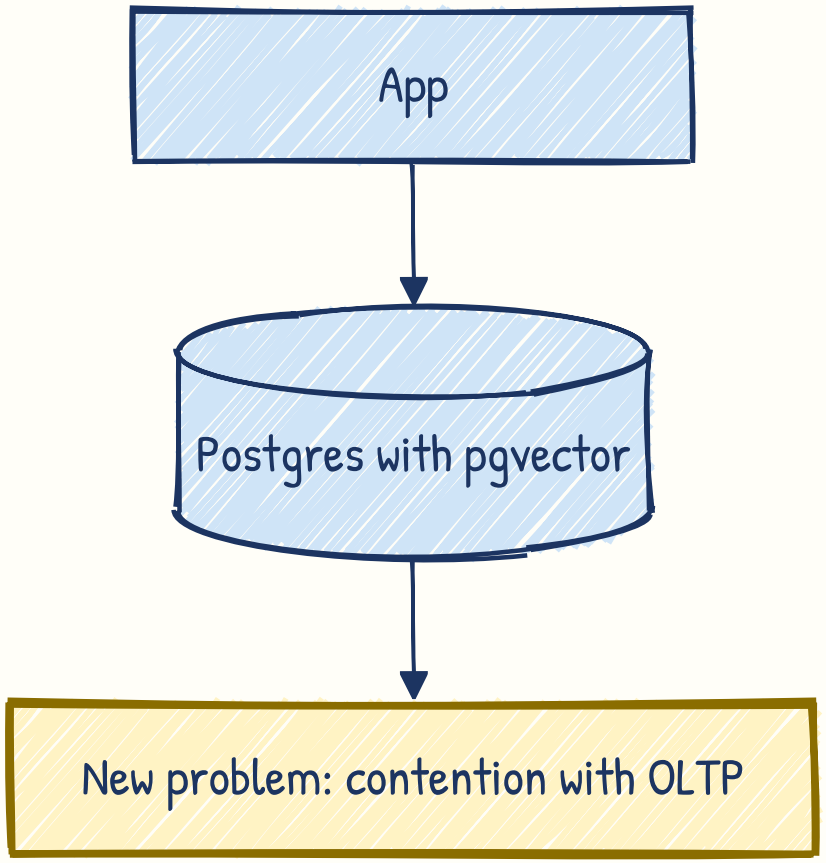


Figure 3. Improved. One system of record, but heavy vector work now shares resources with the transactional load.

Production: a dedicated retrieval database, as a derived index.

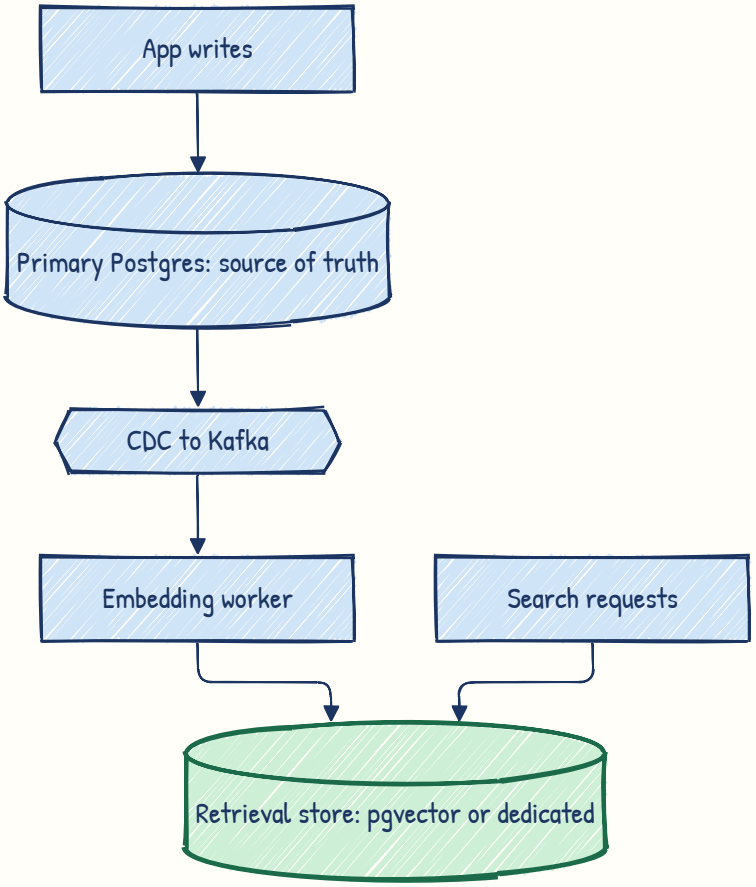


Figure 4. Production. The primary stays the source of truth. A separate retrieval instance is rebuilt from it through change data capture, so its choice of product can change later.

Component	Why it exists	How it communicates
Primary database	The system of record	Application transactions
CDC and Kafka	Moves changes without dual writes (HLD M08, Java 21–22)	Change events
Embedding worker	The only writer to the retrieval store	Consumes events, with idempotent upserts
Retrieval store	Isolates vector load from OLTP	Read by the retrieval service

The trade-off to name: results can lag writes. A deleted document might still be findable for a moment, so **re-check permissions against the primary before showing results** where that matters.

Failures

Failure	Containment
Retrieval store lost	Rebuild it from the primary, since it's a derived index
CDC lag	Alert on lag, and show freshness if needed
Deleted document still returned	Re-check against the source at read time
A managed vendor outage	Degrade to keyword search, or a replica
A pooled connection leaks settings	`SET LOCAL`, and test with a pool

Trade-offs

Decision	Gain	Cost
pgvector	SQL, transactions, one system	Limits at very large scale
Search engine	Mature hybrid search	Cluster operations
Managed service	No operations	Cost and lock-in
Self-hosted dedicated	Features and scale	You run it
Derived index design	Replaceable store	Replication lag

When a dedicated store is the right first choice: you already expect hundreds of millions of vectors, your source data isn't in Postgres anyway, or you've got no database team and want a managed service.

Common mistakes

1. **Comparing feature matrices first.** Start from your constraints.
2. **Benchmarking latency with no recall target.** It's meaningless.
3. **Benchmarking without filters.** Filtered search is where products differ.
4. **Duplicating permissions in a second system.** They'll drift.
5. **Making the vector store your only copy.** Keep it rebuildable.
6. **Using `SET` on pooled connections.** Settings leak across requests.

What to say in an interview

"I'd start from where the data lives and how big it'll get. If it's in Postgres and fits in RAM, I'd use pgvector, because filtering and permissions sit next to the vectors. I'd treat the search store as a rebuildable derived index, so changing it later is a batch job. And I'd benchmark at a fixed recall, with our own filters and writes running."

Certification lens

How this lesson maps to CCAR-F

Not tested. Product selection and vector database details are out of scope.

These are original practice questions for understanding, not exam simulations.

Q Q1 · Component selection · Medium. Choose a store for each. (a) 2,000 FAQ vectors in a Spring Boot app. (b) 5 million chunks with row-level permissions, in Postgres already. (c) A startup with no ops staff, thousands of tenants, and usage-based budgets.

A (a) Exact search in memory, or pgvector with no index. (b) pgvector. (c) A managed service with namespaces, such as Pinecone, after a benchmark. Each choice follows from the constraints, not features.

Q Q2 · Java debugging · Hard. Occasionally, a tenant's search uses another tenant's RLS context. The code runs `SET app.tenant_id = ?` at the start of each request, using HikariCP. What's wrong?

A Plain `SET` persists on the connection, which returns to the pool, so a code path that doesn't reset it can inherit the old value. Use `SET LOCAL` inside a transaction, or `set_config(..., true)`, so that the value ends with the transaction. Add a test that exercises a pool of one connection.

Q Q3 · Error identification · Medium. A vendor report says "4 ms p99 at 10 million vectors". What's missing from this claim?

A The recall target above all, along with filters, write load, hardware, dimensions, concurrency and whether results came from a warm cache. Latency with no stated recall can't be compared.

Revision - Lesson 4

→ Choose by constraints: where data lives, scale, tenancy and who operates it.

→ Start in the database you already run, unless a measured limit says otherwise.

→ The hard part is vectors plus business data, so a new system means a second copy.

→ Leave Postgres on evidence: index outside RAM, tail latency, OLTP contention.

→ A separate retrieval instance is a useful step before a new product.

→ pgvector design: partition by tenant, store model version, hash, text and source.

→ Row-level security adds a database-level guard. Use SET LOCAL on pooled connections.

→ Benchmark on your data, with filters and writes, comparing latency at a fixed recall.

→ Treat the search store as a rebuildable derived index, using CDC.

→ Re-check permissions against the source where lag matters.

Your notes